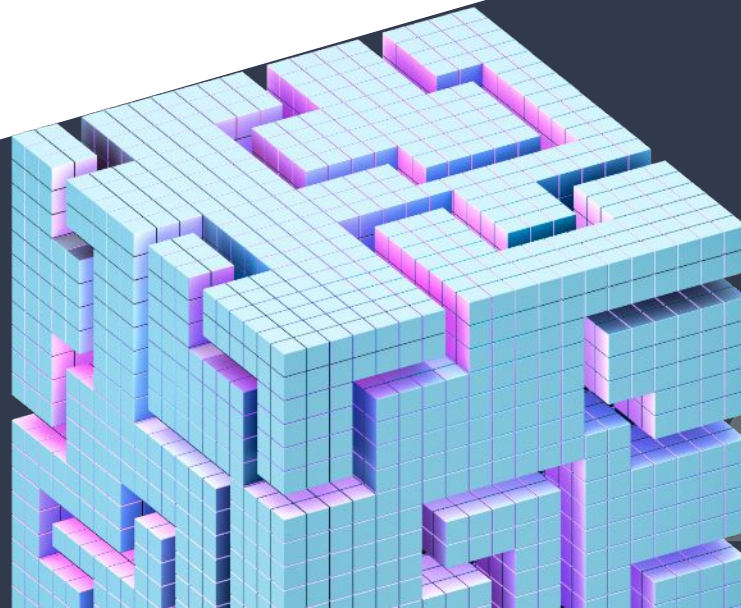


DevOps-Driven API Governance

API Days Munich 2026



Andrzej Jarzyna
Krzysztof Madeński
Munich, July 2026

Andrzej Jarzyna

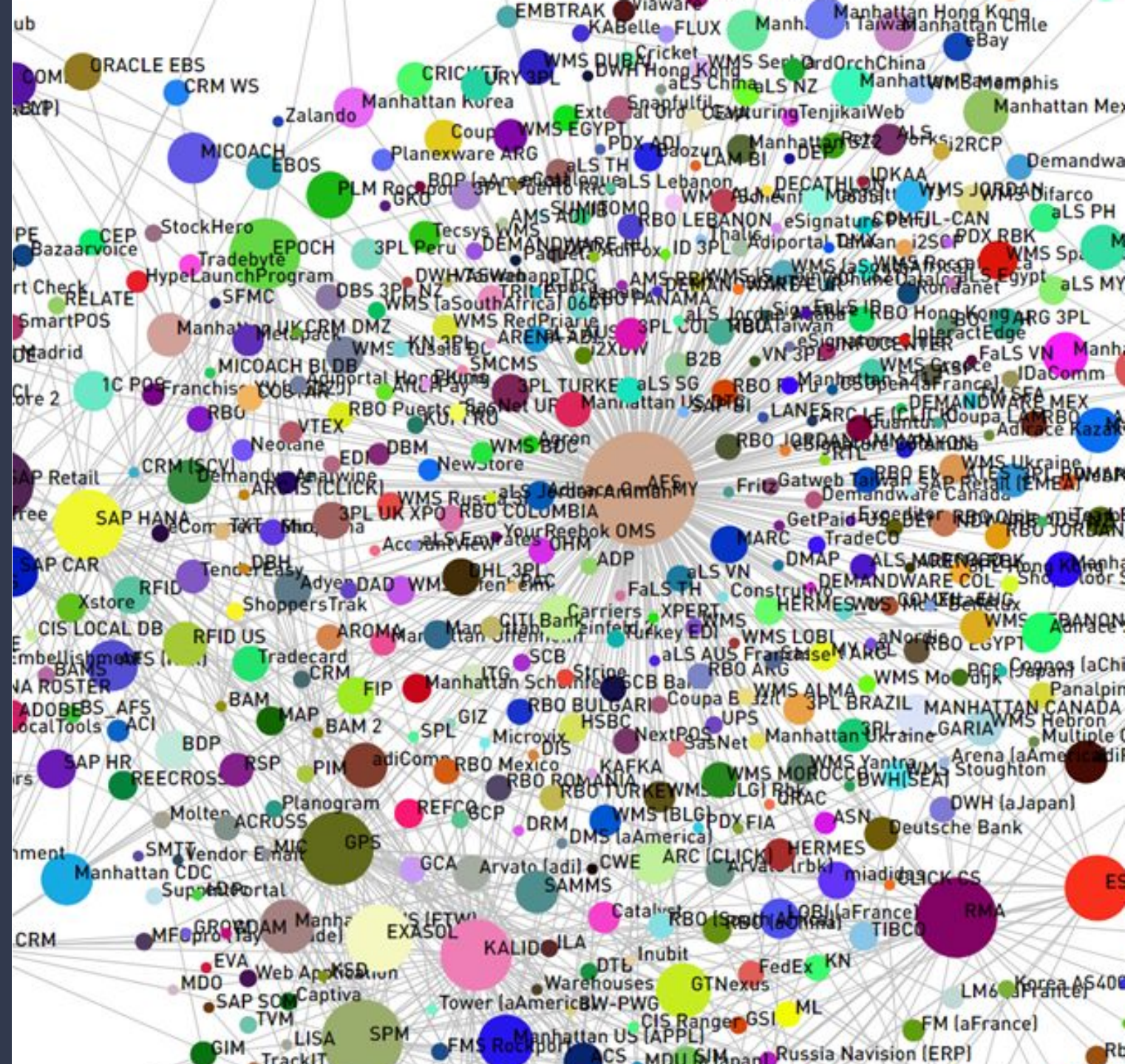


Krzysztof Madeński



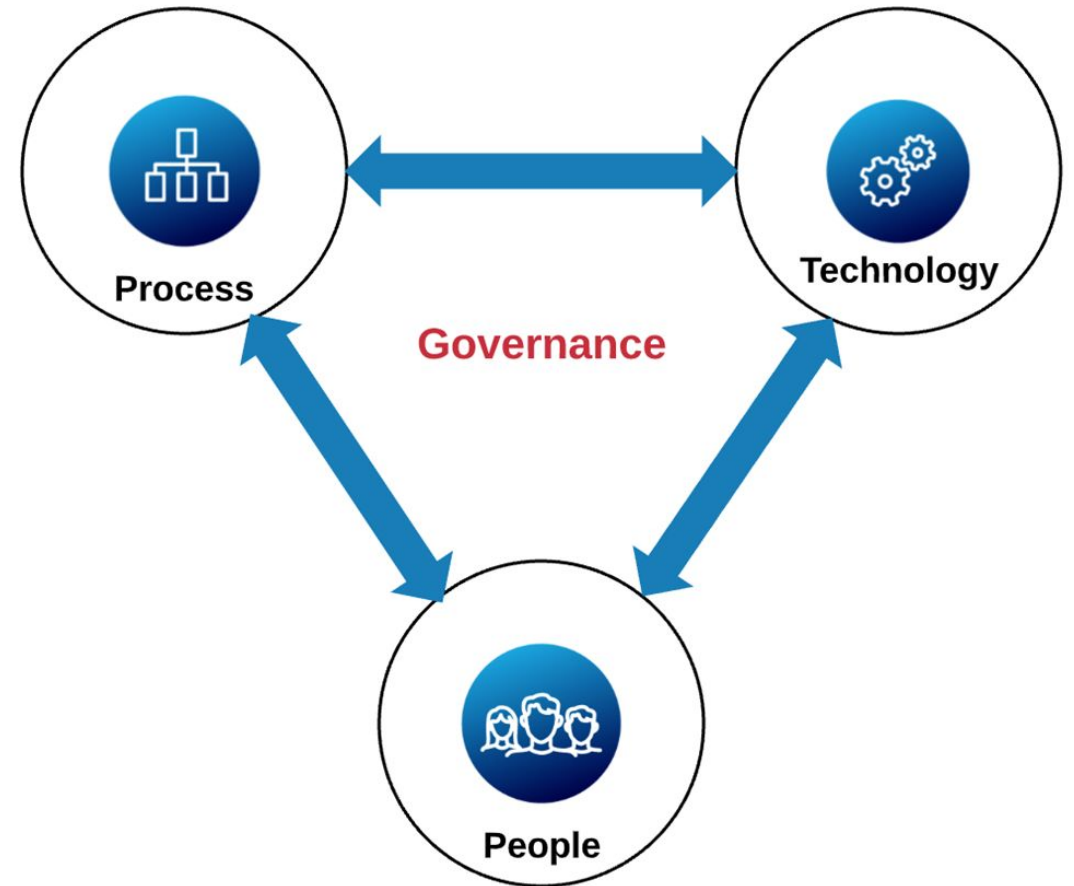
Why do we do Governance?

- Lack of discoverability -> API Sprawl, zombie endpoints
- Manual checks later in the lifecycle
- Uneven quality - leads to tools and agents problems



What API Governance is about?

Make it easy to do things right
and hard to do things wrong



Frictionless

Good governance is invisible to the user

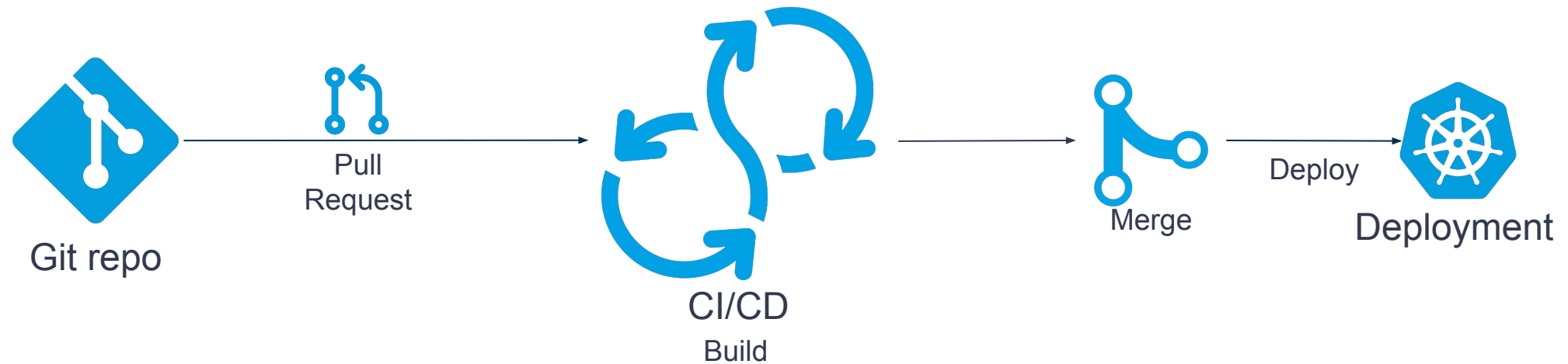
The Devops Process

Premises

- Design-first approach
- API Contract as the single source of truth (OAS/AAS)
- Configuration-as-code
- Github-style CI runner
- <https://github.com/jerzyn/devops-api-governance>



API Governance in the pipeline



You can't govern what you can't see

Topic 1 — API Catalog

thousands of APIs across the org, who knows they exist?

One team's backyard

The screenshot shows a Gitea web interface for a repository named 'governance-demo / devops-api-governance'. The top navigation bar includes 'Explore', 'Help', 'Register', and 'Sign In'. Below the repository name, there are links for 'Code', 'Issues', 'Pull Requests', 'Actions', 'Packages', 'Projects', 'Releases', 'Wiki', and 'Activity'. The repository statistics show 3 commits, 3 branches, and 0 tags. A search bar is present on the right. The main content area displays a list of files and directories, each with a commit message and a timestamp of '3 weeks ago'. The files include '.gitea/workflows', 'contracts', 'sample-backend', '.gitignore', 'catalog-info.yaml', and 'README.md'. The 'README.md' file is expanded, showing the title 'Sample Orders API (consumer repo)' and a description: 'Governed by the PR pipeline in .gitea/workflows/. Contract, provider and catalog-info.yaml are introduced via the first PR.'

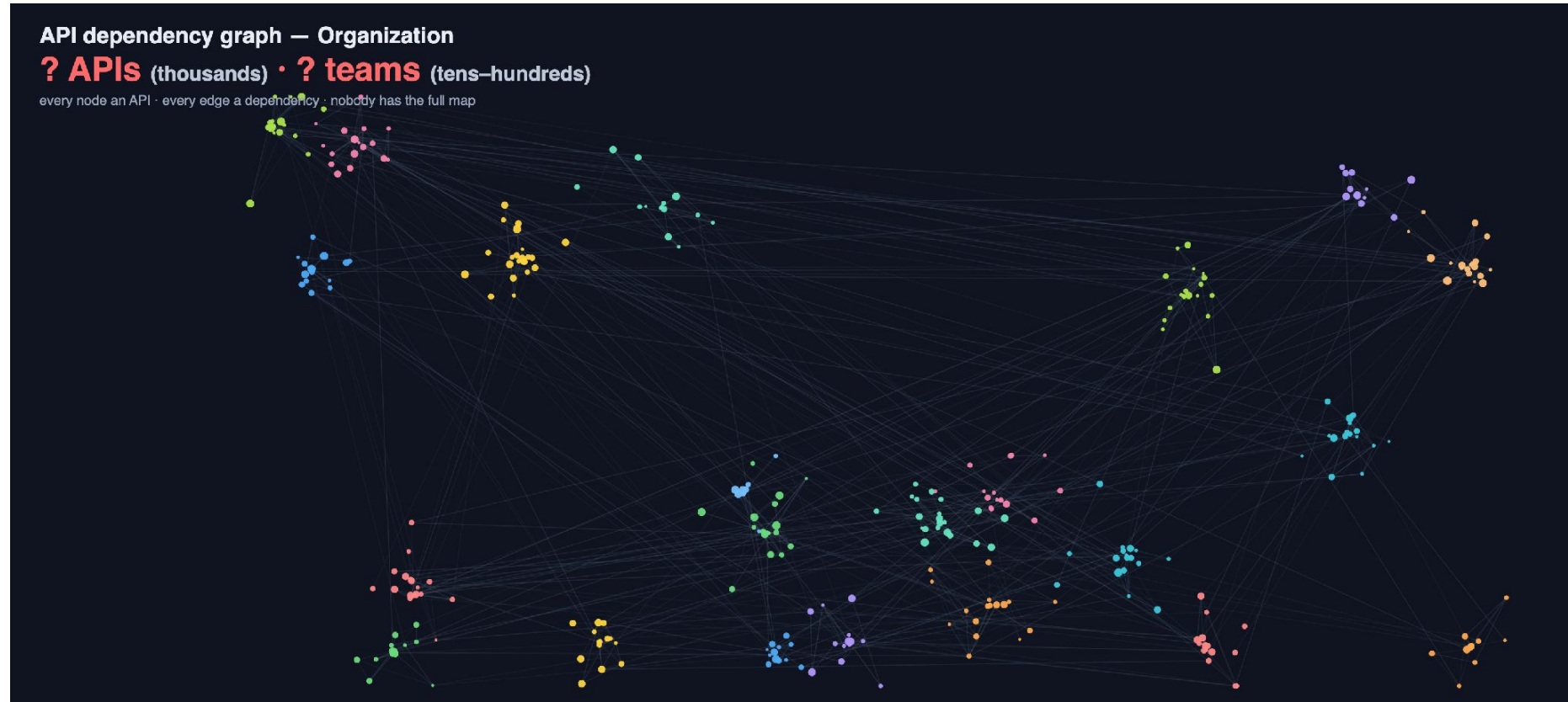
File	Commit Message	Timestamp
.gitea/workflows	chore: bootstrap repo with PR governance pipeline	3 weeks ago
contracts	PR #2 — Evolve to v1.1: add currency	3 weeks ago
sample-backend	PR #2 — Evolve to v1.1: add currency	3 weeks ago
.gitignore	chore: bootstrap repo with PR governance pipeline	3 weeks ago
catalog-info.yaml	PR #1 — Introduce Sample Orders API (contract-first)	3 weeks ago
README.md	chore: bootstrap repo with PR governance pipeline	3 weeks ago

Sample Orders API (consumer repo)

Governed by the PR pipeline in `.gitea/workflows/`. Contract, provider and `catalog-info.yaml` are introduced via the first PR.

- repo, code, OpenAPI, CI on every PR

Now zoom
out – the
whole
organization



- you can't govern what you can't see

One file.
That's the
whole cost.

```
# catalog-info.yaml
apiVersion: backstage.io/v1alpha1
kind: API
metadata:
  name: sample-orders-api
  owner: platform-team
spec:
  type: openapi
  lifecycle: production
  definition:
    $text: ./contracts/orders-openapi.yaml
```

- name, owner, OpenAPI spec - next to the code
- the first step has to be almost **invisible**

Merge it – and you're done

🔗 0 Open

✅ 2 Closed

Label ▾ Milestone

🔗 PR #2 — Evolve to v1.1: add currency (Spectral -> oasdiff -> Microcks) ✓

#2 by demo was merged 3 weeks ago main << feat/add-currency

🔗 PR #1 — Add catalog-info.yaml (register the API in the catalog) ✓

#1 by demo was merged 3 weeks ago main << feat/introduce-orders-api

- commit, merge, **done**
- one commit, a few lines

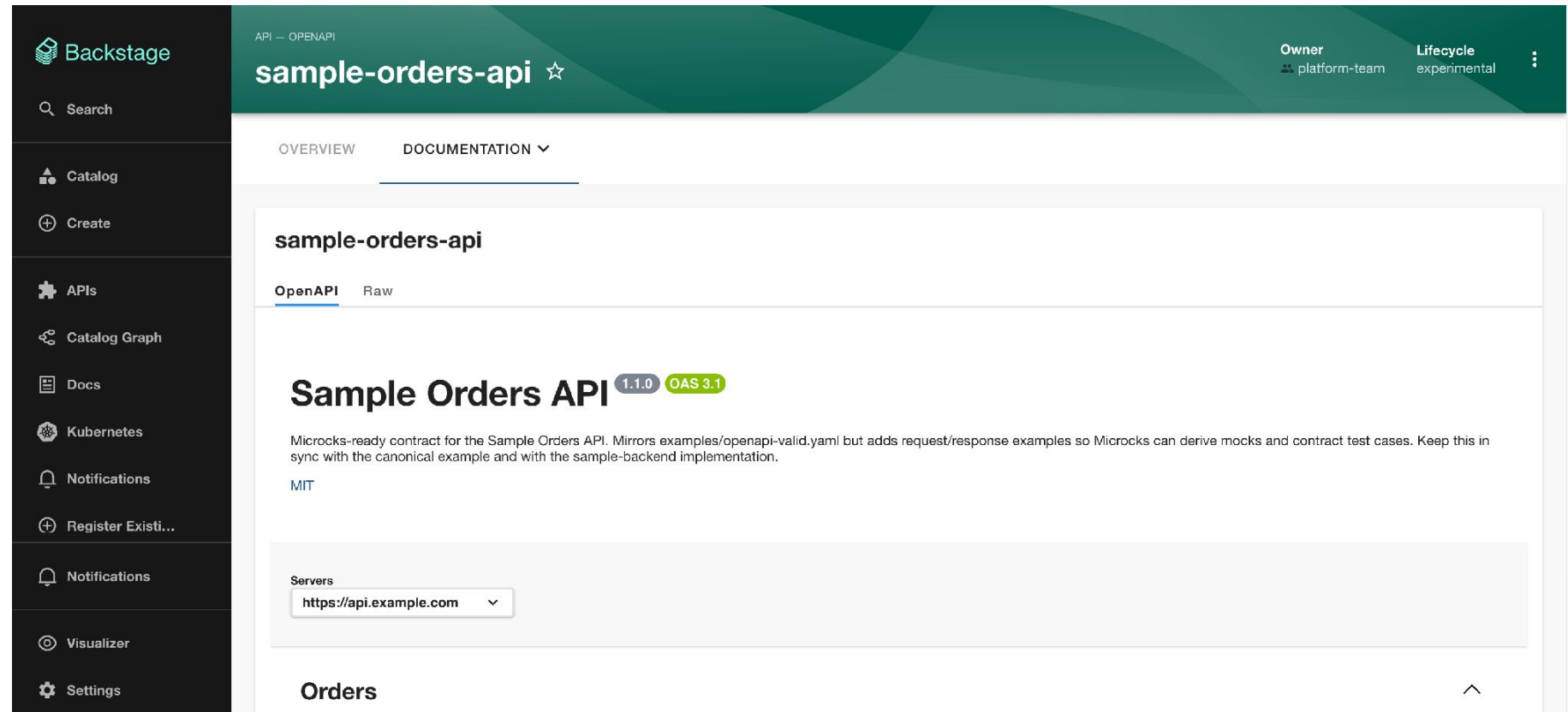
The API appears in the catalog

The screenshot displays the Backstage API Explorer interface. On the left is a dark sidebar with navigation links: Backstage, Search, Catalog, Create, APIs (highlighted), Catalog Graph, Docs, Kubernetes, Notifications, Register Existing..., Notifications, Visualizer, and Settings. The main content area is titled 'My Company API Explorer' and includes a 'REGISTER EXISTING API' button. Below the title, there are filters for 'Type' (set to 'all'), 'PERSONAL' (Owned: 0, Starred: 0), and 'MY COMPANY' (All: 1). Further down are filters for 'Owner' and 'Lifecycle'. The central part of the interface shows a table titled 'All APIS (1)' with a search filter. The table has columns for NAME, SYSTEM, OWNER, TYPE, LIFECYCLE, DESCRIPTION, TAGS, and ACTIONS. One API is listed: 'sample-orders-api' owned by 'platform-team', of type 'openapi' and lifecycle 'experimental'. The description is 'Sample Orders A...'. The tags are 'openapi' and 'orders'. The actions column contains icons for external link, edit, and star.

NAME	SYSTEM	OWNER	TYPE	LIFECYCLE	DESCRIPTION	TAGS	ACTIONS
sample-orders-api		platform-team	openapi	experimental	Sample Orders A...	openapi, orders	[Link] [Edit] [Star]

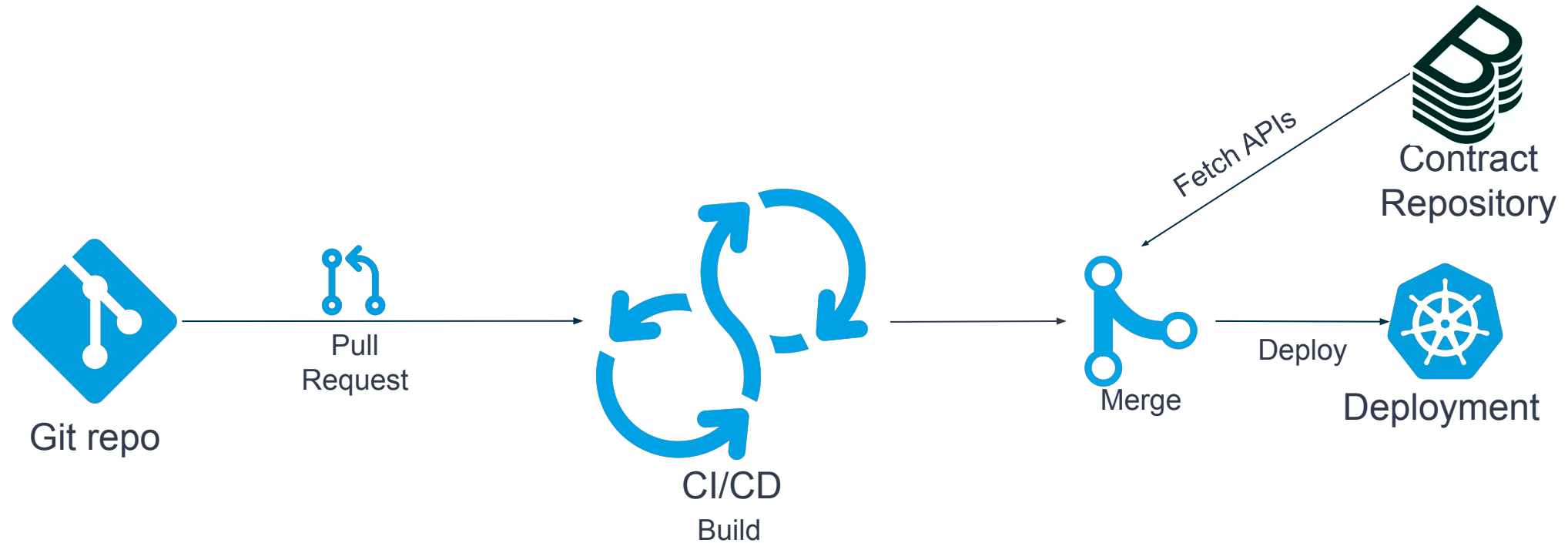
- auto-discovered, org-wide visible, no duplicates

Contract, owner, docs – one place



- contract, owner, docs in one place
- ~~platform migration, runtime change~~
- visibility first

API Governance in the pipeline



A guideline nobody enforces is a suggestion

Step 2 — Guidelines as Code
well written, carefully review but...

Guidelines live in the API catalog

The screenshot shows a web interface for 'API Guidelines'. At the top, there's a 'Documentation' header. Below it, the title 'API Guidelines' is displayed on the left, and a search bar 'Search docs' is on the right. A metadata bar indicates 'Component: api-guidelines', 'Owner: platform-team', and 'Lifecycle: production'. The main content area is titled 'General API Guidelines' and contains a section 'API First Approach (pzu:general1:2025-api-first)' with a paragraph explaining the principle. Below this is a section 'Language (pzu:general2:2025-language)' which currently shows 'TBD.'. On the right side, there is a 'Table of contents' with links to various sections: 'API First Approach', 'Language', 'Terminology', 'New and Existing APIs', 'Semver', 'Version Elements', 'Contract', 'Robustness', and 'Version Control System'.

Documentation

API Guidelines

Search docs

Component: [api-guidelines](#) Owner: [platform-team](#) Lifecycle: production

API Guidelines

[API Guidelines](#)

General API Guidelines

API First Approach (pzu:general1:2025-api-first)

Everyone **SHOULD** follow the **API First** principle. The API First principle is an extension of the **design-first** principle. Therefore, API development **SHOULD** always begin with the API design without any prior coding activities. The API design (e.g., description, schema) is **the source of truth**, not the API implementation. The API implementation **MUST** always conform to the specific API design, which represents the contract between the API and its consumer.

Language (pzu:general2:2025-language)

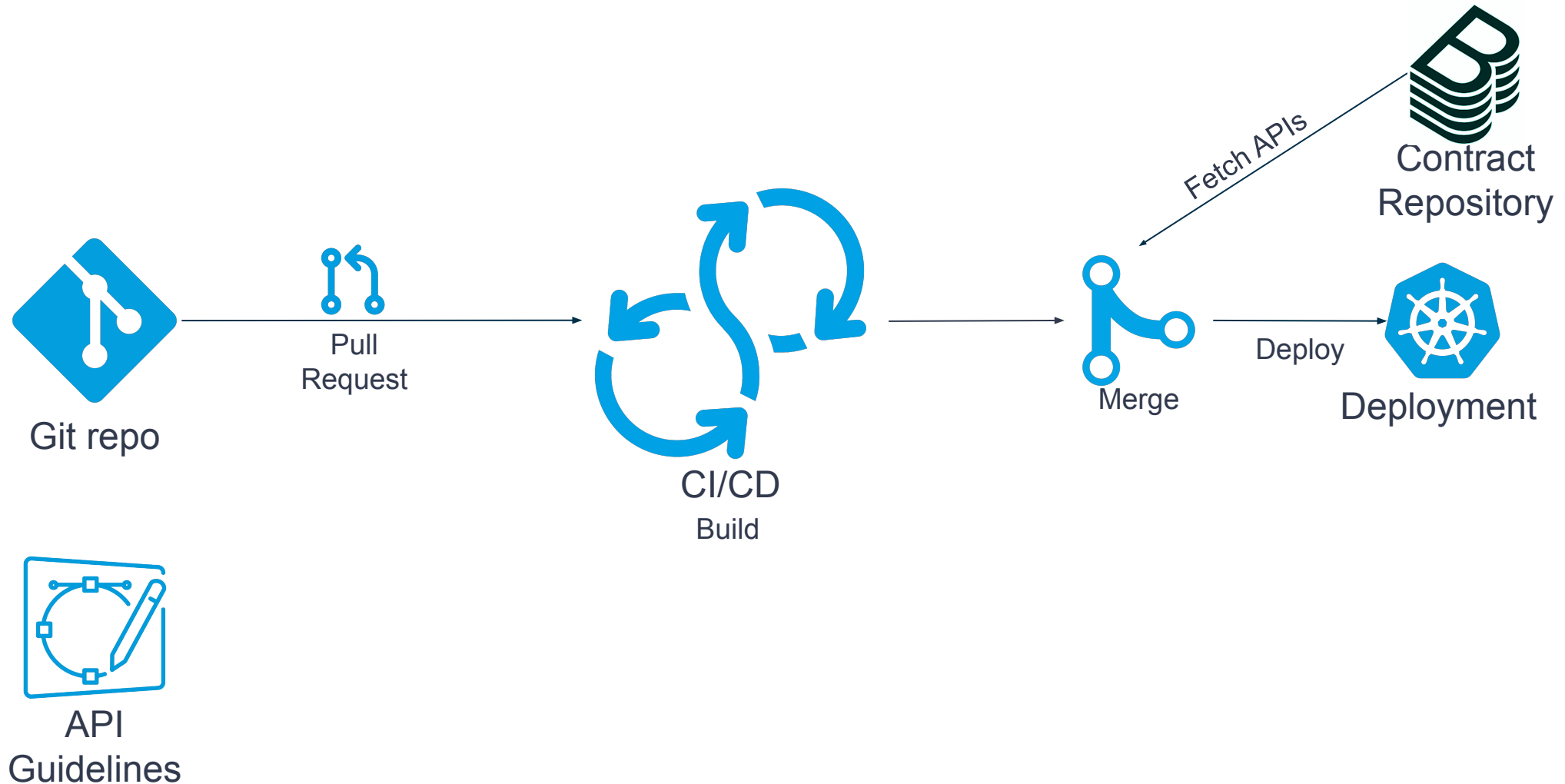
TBD.

Table of contents

- [API First Approach \(pzu:general1:2025-api-first\)](#)
- [Language \(pzu:general2:2025-language\)](#)
- [Terminology \(pzu:general3:2025-terminology\)](#)
- [New and Existing APIs \(pzu:general4:2025-new-existing-APIs\)](#)
- [Semver \(pzu:general5:2025-semver\)](#)
- [Version Elements](#)
- [Contract \(pzu:general6:2025-contract\)](#)
- [Robustness \(pzu:general7:2025-robustness\)](#)
- [Version Control System \(pzu:general8:2025-version-control\)](#)

- markdown in the catalog – readable with examples
- same rules for humans & AI – one source of truth

API Governance in the pipeline



The same rules – as code

```

v [x] Run Spectral (fail on errors)
17 spectral contracts/orders-openapi.yaml
18
19 /workspace/governance-demo/devops-api-governance/contracts/orders-openapi.yaml
20 2:6 warning info-contact Info object must have "contact" object.
    info
21 14:10 warning oas3-server-not-example.com Server URL must not point at example.com.
    servers[0].url
22 16:5 warning tag-description Tag object must have "description".
    tags[0]
23 20:9 error pzu:rest13:2025-operation-name-required Operation MUST have a 'summary' field.
24 https://api-guidelines.app.pzu.pl/rest/#nazwa-operacji-pzurest132025-operation-name`
    paths./orders.get
```

- executable, deterministic
- operation summary is missing?
- PR blocked -> rule, file, line, how to fix feedback

Fix. Push. Green.

The screenshot shows the Gitea Actions interface for a repository named 'governance-demo / devops-api-governance'. The 'Actions' tab is selected, displaying a workflow run for 'PR1 step 3/3: implement GET /orders in provider (contract-first) -> ALL GREEN'. The workflow was triggered via a pull request and is in a 'Success' state with a total duration of 27s. The workflow consists of three jobs: 'spectral-openapi-check' (15s), 'breaking-changes-check' (4s), and 'contract-test' (4s). The 'Summary' section on the left lists these jobs with their respective durations. The 'Workflow Dependencies' section on the right shows a linear sequence of the three jobs. The footer indicates the system is powered by Gitea version 1.26.2, with a page load time of 16ms and a template load time of 6ms. The footer also includes links for 'Auto', 'English', 'Licenses', and 'API'.

governance-demo / devops-api-governance

<> Code Issues Pull Requests **Actions** Packages Projects Releases Wiki Activity Settings

✓ PR1 step 3/3: implement GET /orders in provider (contract-first) -> ALL GREEN

pr-governance.yml: Commit f621b6894f pushed by demo #1

Re-run all jobs

Summary

All jobs

- ✓ spectral-openapi-check 15s
- ✓ breaking-changes-check 4s
- ✓ contract-test 4s

Run Details

Workflow file

Triggered via pull_request • 2 weeks ago

Success • Total duration: 27s

Workflow Dependencies

3 jobs • 2 dependencies • 100% success

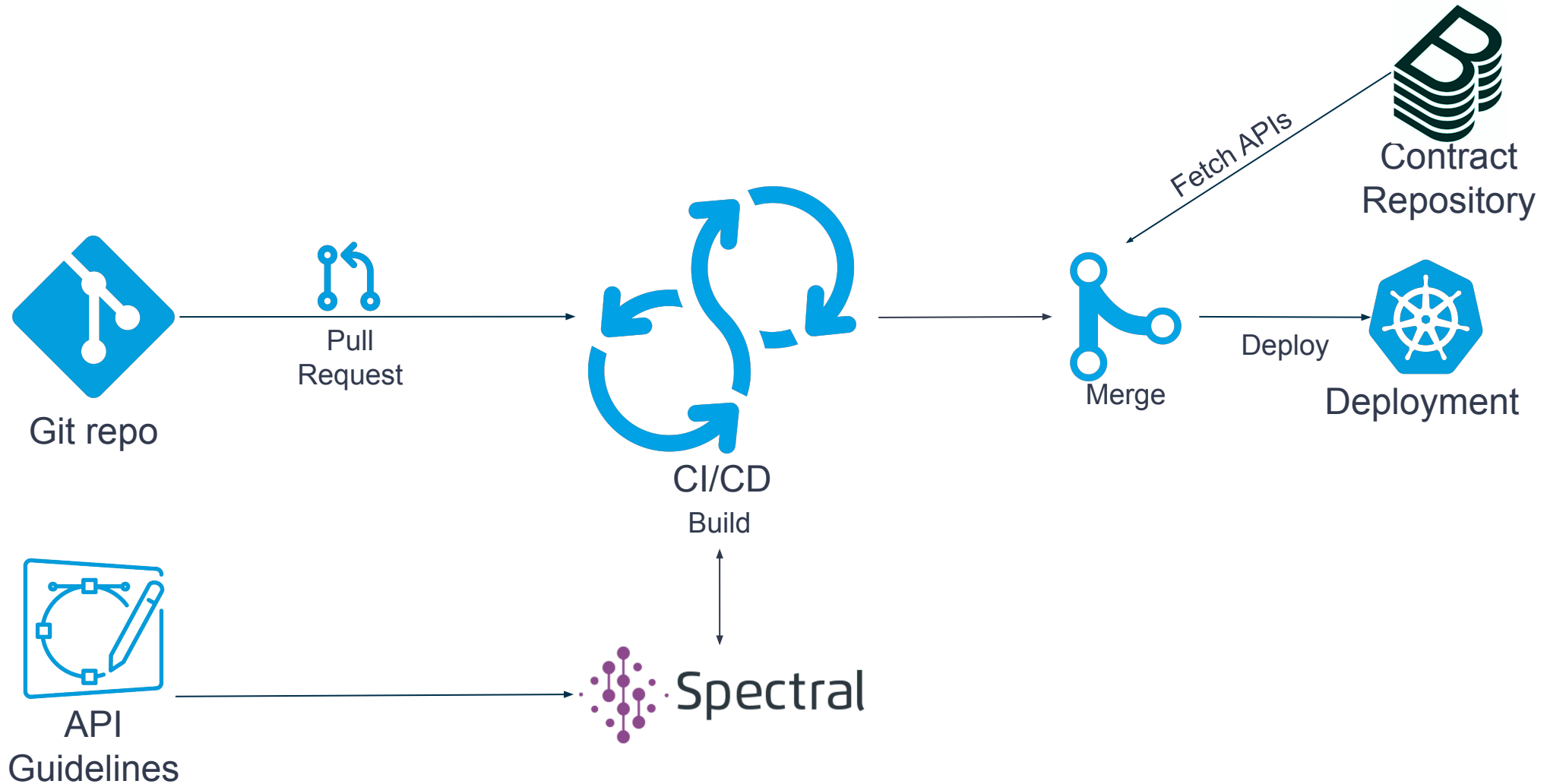
spectral-openapi... 15s breaking-changes... 4s contract-test 4s

Powered by Gitea Version: 1.26.2 Page: 16ms Template: 6ms

Auto English Licenses API

- any CI, no lock-in
- feedback in seconds, not weeks – right at PR

API Governance in the pipeline



Iterate on the API before writing a line of code

Step 3 — Central Mocking & Contract Testing

a contract is a promise

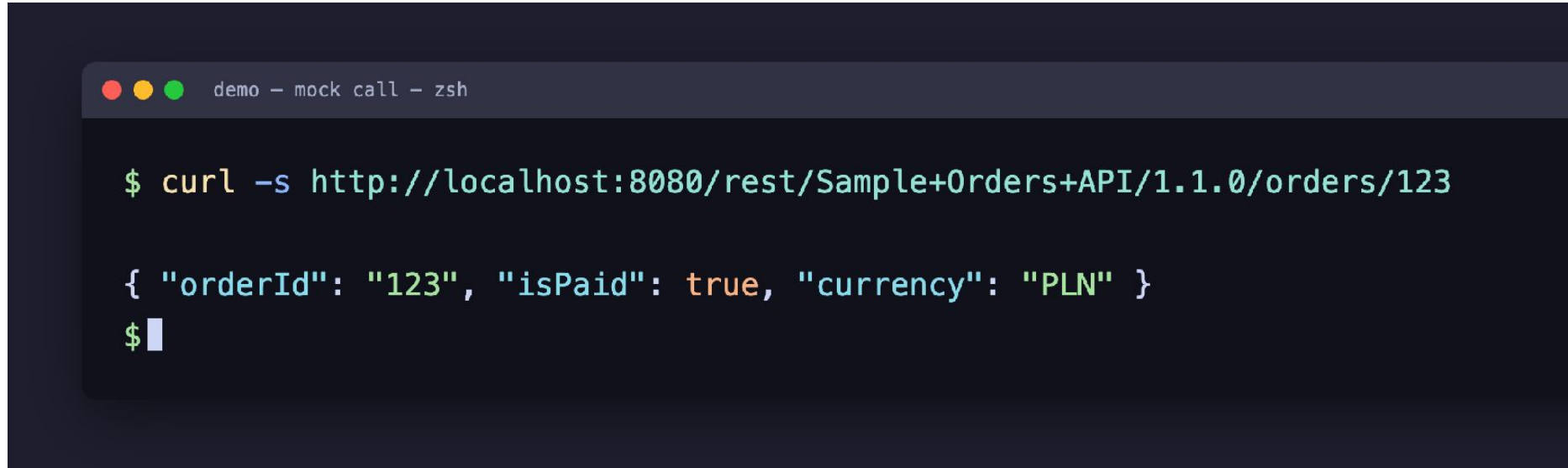
Push the
spec → get a
live mock

Q1 — integrate before it exists?

The screenshot displays the Microcks web interface. On the left is a dark sidebar with navigation links: Dashboard, APIs | Services (active), Importers, Live Traces, Microcks Hub, and Administration. The main content area is titled 'Sample Orders API - 1.1.0' and includes a breadcrumb 'Services & APIs > Sample Orders API - 1.1.0'. Below the title, it shows the creation date and an 'Edit Labels' link. The 'PROPERTIES' section lists the Service Type as REST, the Contract as 'Sample Orders API-1.1.0.yaml OPEN_API_SPEC', and provides links for View Documentation, Mocks Invocations, HTTP SSE Endpoint, and HTTP Streamable Endpoint. On the right, a 'Tests' panel shows a Conformance index bar (A, B, C, D, E) with 'C' highlighted, a Conformance score of 25.0%, and a '+ New Test' button. The 'OPERATIONS' section at the bottom lists two endpoints: 'GET /orders' and 'GET /orders/{orderId}', both with a 'GET' method and '1 sample(s)'.

- just OpenAPI file -> org-wide live mock

The mock responds – partners integrate today

A terminal window with a dark background and light-colored text. The title bar at the top shows three colored circles (red, yellow, green) followed by the text "demo - mock call - zsh". The terminal content shows a command being executed: "\$ curl -s http://localhost:8080/rest/Sample+Orders+API/1.1.0/orders/123". Below the command, the output is a JSON object: "{ \"orderId\": \"123\", \"isPaid\": true, \"currency\": \"PLN\" }". The prompt "\$" is visible at the end of the output line.

```
demo - mock call - zsh

$ curl -s http://localhost:8080/rest/Sample+Orders+API/1.1.0/orders/123

{ "orderId": "123", "isPaid": true, "currency": "PLN" }
$
```

- real response from contract.examples
- iterate with clients before implementation
- contract-first consumer-driven

Then we
implement
– and drift
gets caught

Contract
Testing

Q2 – does implementation do what the contract says

Q2 — does code match the contract?

RESULTS

>

GET /orders

1 test(s)

in 5 ms

▼

GET /orders/{orderId}

1 test(s)

in 1 ms

order_123

Request

Header name	Values
Accept	application/json

Response

Response Time:

1 ms.

Response Code and Type:

200: application/json

Error:

required property 'currency' not found

{"orderId": "123", "isPaid": true}

- spec promised a required field, code didn't
- API drift caught automatically

In sync – contract & code

```
# one example inside the OpenAPI contract...
```

```
examples:
```

```
  order_123:
```

```
    value:
```

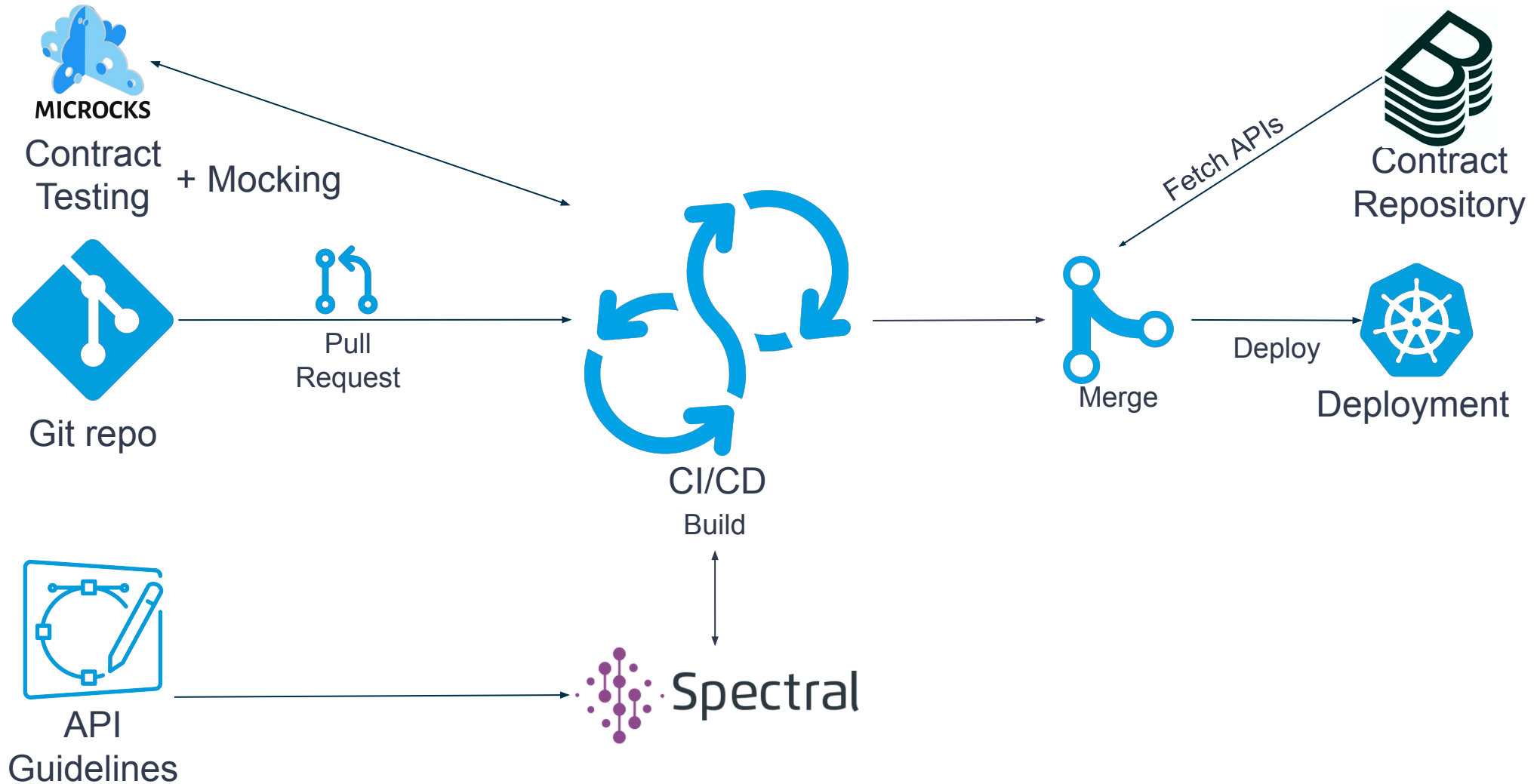
```
      orderId: "123"
```

```
      isPaid: true
```

```
      currency: "PLN"
```

- example -> mocks , example -> test - double return
- + generic MCP mock from OpenAPI

API Governance in the pipeline



Now imagine a partner is integrated

Step 4 — Breaking Changes

The innocent change

orders-openapi.yaml — the "innocent" change

Order:

required:

- orderId
- customerId # ← was optional, now REQUIRED

properties:

orderId: { type: string }

customerId: { type: string }

currency: { type: string } # ← new field

- add currency – safe, addictive
- make customerId **required** – innocent?

Blocked – before anyone gets hurt

```
$ oasdiff breaking base → PR (orders-openapi.yaml)
```

```
ERROR request-property-became-required
```

```
POST /orders · request property 'customerId' became required  
clients that omit it today will start getting 400
```

```
1 breaking change · exit code 1 → PR blocked
```

- optional -> required = breaking
- quest checkouts/internal tools -> 4xx
- before merge, before deploy, before incident

An explicit decision — not an accident

The screenshot shows a GitHub Actions workflow run for the repository 'governance-demo / devops-api-governance'. The workflow is titled 'PR2 step 4/4: provider returns currency (contract-first) -> ALL GREEN' and was triggered by a pull request. The status is 'Success' with a total duration of 22s. The workflow consists of three jobs: 'spectral-openapi-check' (10s), 'breaking-changes-check' (4s), and 'contract-test' (4s). The 'Workflow Dependencies' section shows a linear sequence of these three jobs, all of which passed successfully. The 'Summary' section lists the jobs and their durations. The 'Run Details' section provides a link to the workflow file.

governance-demo / devops-api-governance

PR2 step 4/4: provider returns currency (contract-first) -> ALL GREEN

pr-governance.yml: Commit ba189344de pushed by demo #2

Summary

All jobs

- ✓ spectral-openapi-check 10s
- ✓ breaking-changes-check 4s
- ✓ contract-test 4s

Run Details

Workflow file

Triggered via pull_request • 2 weeks ago

Success • Total duration: 22s

Workflow Dependencies

3 jobs • 2 dependencies • 100% success

spectral-openapi... 10s

breaking-changes... 4s

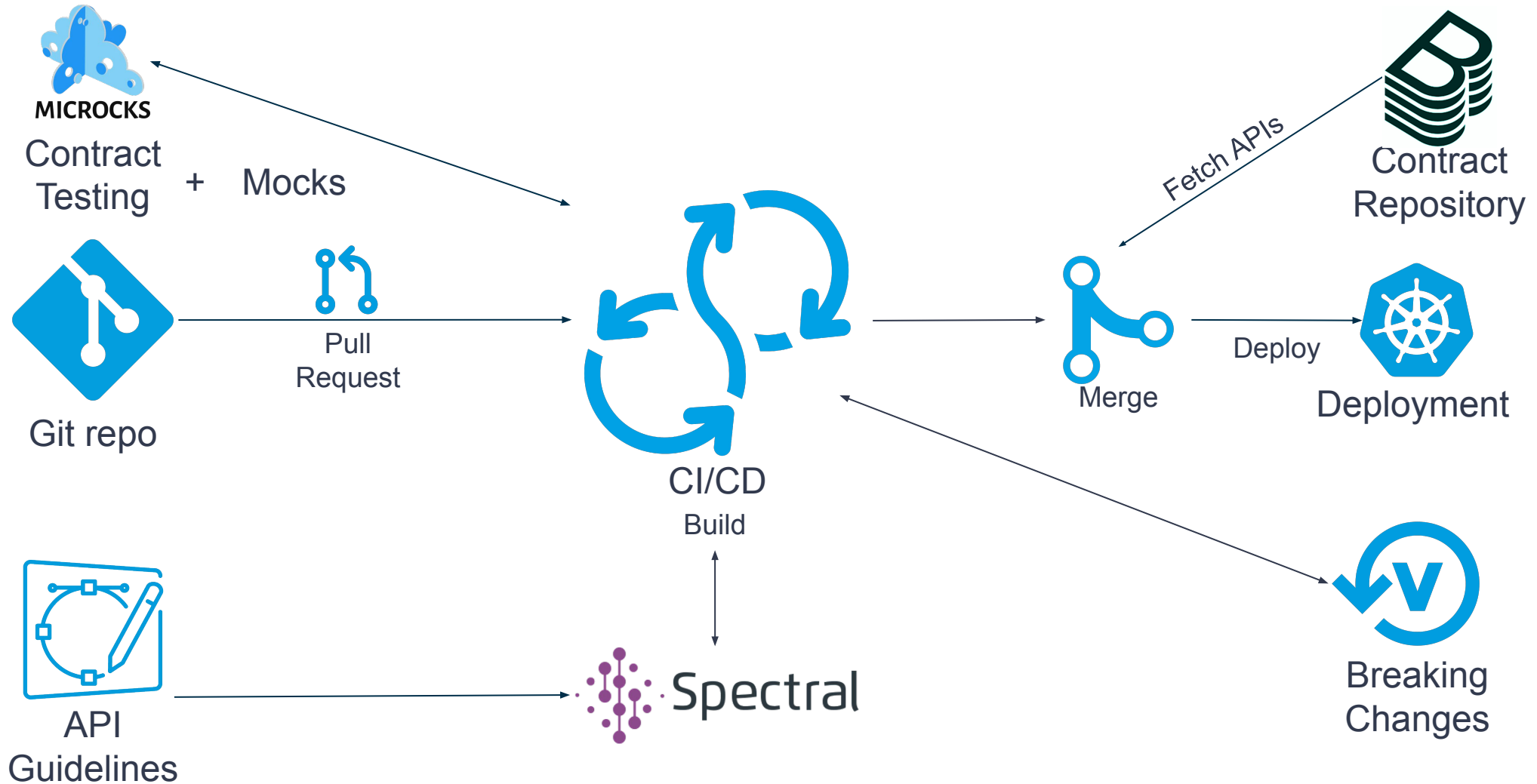
contract-test 4s

Powered by Gitea Version: 1.26.2 Page: 10ms Template: 3ms

Auto English Licenses API

- decide on purpose
- no breaking by accident

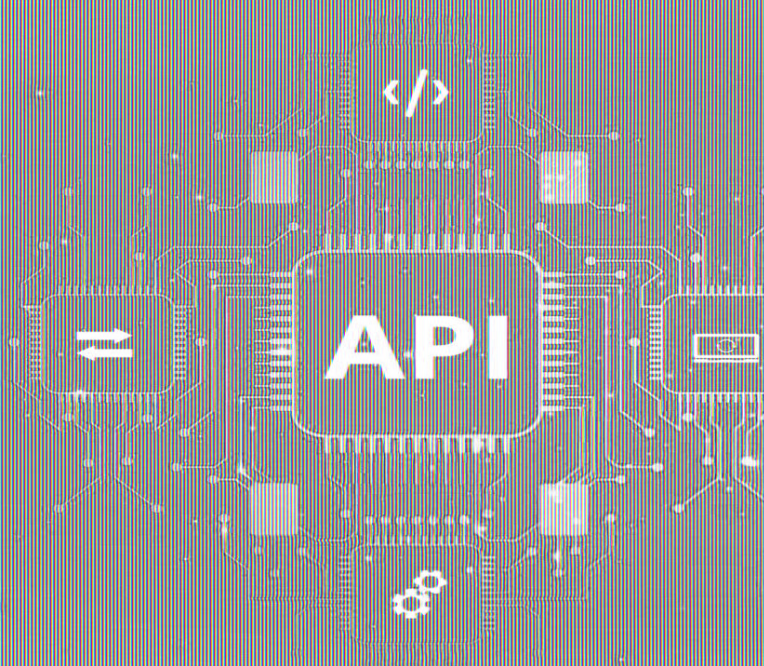
API Governance in the pipeline



RESTful API

Design Patterns and Best Practices

Master REST API design with real-world examples, lifecycle management, and OpenAPI specifications



Andrzej Jarzyna | Samir Amzar



Share your feedback about speech:

Thank you!

